

面向生产的 Agent 架构、工具封装与工具间编排深度调研报告

执行摘要

Agent 在工程语境中通常指“能够在护栏（guardrails）内，借助工具获取上下文并采取行动，从而相对自主地完成任务的系统”。这一表述与 OpenAI ¹ 在其开发者资料中对 Agents 的定义一致：Agent 使用大模型执行指令与决策、调用工具补足训练数据之外的信息并采取行动，同时应运行在清晰的护栏之内。²

从 2020-2026 的学术与工业资料看，Agent 架构从“单循环（single loop）”逐步走向“可恢复、可观测、可治理的长时工作流（durable, observable, governable long-running workflows）”。以工具使用（tool/function calling）为核心能力的系统，已形成较稳定的技术底座：以 JSON Schema / 严格结构化输出为工具契约、以调用链追踪与预算控制保证可运维性、以工具面（tool surface）管理与延迟加载减小上下文与成本压力。³

本报告的关键结论与建议如下（面向 C++ 团队可直接落地）：

单 Agent 循环仍是 v1 的最优点：用“工具调用循环 + 严格工具契约 + 预算/权限/观测”即可覆盖大量业务型场景；其思想与 ReAct（Reason+Act 交错）范式一致，能在可控复杂度下显著降低幻觉与错误传播。⁴

当任务呈现长链路与强结构时，应尽快引入“规划-执行分离（planner/executor）”或“图/状态机（graph/state machine）”以提升确定性与可恢复性。LangGraph 强调 durable execution、human-in-the-loop 与 memory，代表了工业界把 Agent 视为“可恢复状态机/工作流”的趋势；Temporal 的 durable execution 语义则为长时、可重试的执行提供成熟范式（尤其适合把工具调用当作 Activities）。⁵

工具封装的核心目标不是“让模型能调用函数”，而是把工具变成可治理的 API：清晰输入输出、错误与超时语义、幂等与重试策略、鉴权与审计、指标与追踪、缓存与版本化。OpenAI 的 function/tool calling 文档明确指出工具定义会计入上下文与计费、模型可能一次返回多个 tool_calls、并建议保持初始可用工具数量较少（软建议 <20），必要时用 tool_search 延迟加载。⁶

允许“工具调用工具”在工程上可行，但必须区分三种形态：固定编排（fixed orchestration）、代码级组合（programmatic composition）、元工具/代码执行编排（meta-tools，例如让模型写编排代码再调用工具）。Anthropic ⁷ 提出的 Programmatic Tool Calling 代表第三类：让模型生成 Python 编排逻辑，把中间结果留在执行环境里，仅把最终摘要进入模型上下文，从而减少 token 污染与多轮推理开销，并支持工具并行。⁸

风险治理优先级应高于“更强自治”：递归与死循环、调用链追踪困难、成本/延迟不可控、调试与复现困难、安全边界突破，是工具互调最大风险面。缓解手段应体系化：预算（token/成本/时间/步数/深度）、白名单与 allowed_callers、并行开关（如 parallel_tool_calls）、强观测（tracing + call graph）、以及面向 LLM 应用的安全基线（OWASP Top 10 for LLM Applications、NIST AI RMF）。⁹

建议的工程里程碑（原型到可灰度上线）通常可按 6-10 周拆分：v1 单 Agent + 工具契约 + 观测；v2 引入 planner/executor 与工具路由；v3 图/状态机与 durable execution；v4 多 Agent（manager/handoff）与更强治理。该节未给出可直接复用的甘特图。 10

Agent 架构定义与分类

Agent 的工程定义可以用“闭环”来刻画：模型在多轮对话/推理中，根据当前状态选择行动（调用工具、读写记忆、请求人类介入等），把外部世界的反馈再写回状态并继续推进，直到满足终止条件。OpenAI 的工具调用流程明确展示了这种“多步对话 + 反复工具调用 + 返回最终回答”的闭环结构。 11

从学术与工业资料可归纳出五类常见架构（实际系统常为混合体），下面给出定义、优劣与典型用例，并在后文给出可落地蓝图。 12

架构对比表

架构类型	核心机制	优点（工程角度）	缺点/风险	典型用例	运维复杂度
单 Agent（single-agent loop）	单个策略/对话循环：模型决定是否调用工具并迭代	最快落地；端到端链路短；适合从 0 到 1	长任务稳定性差；易“走偏”或反复调用；复杂流程难复现	客服/知识问答、数据查询助手、轻量自动化	低 → 中
规划-执行分离（planner/executor）	先规划再逐步执行；必要时重规划	结构化强、可解释；更容易做预算与审计	规划质量不稳会拖慢；过度规划增加 token 与延迟	报告生成、研究类任务、数据管道编排	中
管理者-多 Agent（manager + multi-agent）	manager 拆解/分派；专职 agent 执行并汇总	专业化与并行；更好的人机协作与职责分离	协调成本高；状态一致性与“扯皮”难题	复杂研发协作、运营流程、跨域任务	中 → 高
流水线（pipeline）	固定阶段：检索 → 分析 → 行动 → 校验 → 输出	可预测、易测试、易 SLA；组件可替换	灵活性差；遇到新分支需要改流程	结构化文档处理、RAG workflow、合规审核	中
事件驱动/持久化执行（event-driven/durable）	以事件推进状态机/工作流；支持恢复、长时运行	最强可靠性；天然适配重试/补偿；便于人类介入	平台与工程投入大；需要严格状态建模	长时任务、跨系统自动化、业务流程编排	高

表中机制的代表性材料包括：ReAct 把“推理轨迹与行动（工具/环境交互）交错生成”作为通用范式；AutoGen 把多 Agent 对话作为可编程的协作基础设施；LangGraph 把 Agent 作为长时有状态图并强调 durable execution 与

human-in-the-loop；Haystack 明确展示 agent 可以把“组件与 pipeline”当工具使用；Temporal 的 durable execution 提供长时可恢复执行的工程语义。¹²

单 Agent 与 ReAct 式循环

ReAct 论文提出把“Reasoning traces（推理轨迹）”与“Actions（行动）”交错生成，行动可通过外部 API/环境获取信息并更新计划，从而改善纯推理链的幻觉与错误传播，并提升可解释性。¹³

在工业落地中，OpenAI 的 function calling “五步循环”（给工具→模型返回 tool call→应用执行→把结果回传→得到最终回答或更多 tool calls）基本就是 ReAct 的工程化版本；并且文档明确提示模型一次可能返回 0/1/多条 tool_calls，因此执行器必须按“批量工具调用”来实现。¹¹

适用场景：任务链路短、可接受少量试错、工具集有限且明确；尤其适合先把“工具契约与治理”打牢。¹⁴

规划-执行与更强规划能力

规划-执行思想既可以通过提示词实现（如 Plan-and-Solve 先生成计划再逐步解题），也可以通过框架/运行时以显式结构实现。Plan-and-Solve 提出“先拆分为子任务计划，再按计划执行”以减少缺步错误，并在多数据集上提升零样本推理表现。¹⁵

在框架层面，Semantic Kernel 明确把“自动 function calling”作为一种自动规划循环，并指出其旧式 stepwise/handlebars planners 已弃用，建议新 agent 直接使用 function calling（更灵活、内置工具支持、更简单）。¹⁶

适用场景：需要稳定拆解、多步可审计、以及对“执行顺序/约束”敏感的任务（如合规、财务、操作类）。¹⁷

多 Agent：管理者与对话式协作

AutoGen 把“多 Agent 可对话（conversable agents）并能结合工具与人类输入”作为通用基础设施，允许用自然语言或代码定义交互模式，并在多领域任务中展示效果。¹⁸

多 Agent 的价值通常来自三点：专业化分工（不同提示/工具/权限）、并行化（降低 wall-clock）、以及更强的自检/互审机制。但真实工程里，多 Agent 的成本往往不是推理，而是“协调、状态一致性、冲突解决与可观测性”，因此建议在 v1 先以单 Agent + 强治理跑通，再逐步引入 handoff/manager。¹⁹

流水线与组件化 Agent

流水线的关键不是“是否有模型”，而是“流程结构固定且可测试”。Haystack 教程明确展示两种方式：Agent 直接使用简单工具，或把更复杂的 pipeline（多个组件构成）作为工具来用，从而形成可预测的阶段式结构。²⁰

适用场景：文档处理/RAG、抽取-校验-写回、以及需要强 SLA 和可回归测试的业务流程。²¹

事件驱动与可恢复执行

LangGraph 将 agent 建模为“长时、有状态、可部署的图/状态机”，并强调 durable execution、human-in-the-loop、memory 与生产级部署能力，这与“事件驱动、可恢复运行时”的方向高度一致。²²

Temporal 则从分布式系统角度给出 durable execution (crash-proof execution) 的工程语义，并通过可重试、可恢复的 Activities/Workflows 机制处理长时任务；其文档强调 Activities 可能被重试，因此业务逻辑应幂等（并建议使用 idempotency keys 识别重复请求）。这些机制与 Agent 的“工具调用可重试、可恢复”天然契合。 ²³

需要特别指出：即使是“擅长工具使用”的前沿模型，在长时一致性与鲁棒性上仍存在缺口。ToolHaystack 的系统评测指出，即便具备长上下文能力的模型，在长程场景下仍可能出现一致性与鲁棒性问题，暗示真实部署必须依赖架构性护栏而非仅靠模型能力。 ²⁴

工具封装的工程设计模式

工具封装 (tool encapsulation) 的目标是把外部能力变成“可治理、可观测、可演进”的接口层，而不是把一段函数暴露给模型。OpenAI 明确把 function/tool 定义为“我们给模型的功能”，并把工具调用视为“模型发出的特殊请求”，要求应用端执行并把输出以 call_id 关联回去。 ¹¹

本节给出一套可直接用作设计评审 checklist 的工具契约模式（适用于自研工具、HTTP API、DB、队列、文件系统、脚本等），并穿插来自 OpenAI、Gemini、MCP、Temporal 等资料的关键约束。 ²⁵

工具接口契约：Schema、I/O 与严格模式

一等公民：JSON Schema 作为工具输入契约

OpenAI 将 function 定义为“由 JSON schema 定义的工具”，并在 strict mode 下要求 schema 更严格（例如对象需 additionalProperties=false 且 properties 全 required），并建议始终开启 strict 以提高函数调用的可靠一致性。 ²⁶

调用关联：call_id / id 必须贯穿全链路

OpenAI 的工具输出需要引用特定工具调用（示例以 call_id 关联）。Gemini 文档也强调为每次 function call 生成唯一 id，并建议在回传 functionResponse 时带上匹配 id（SDK 可自动处理）。这意味着运行时层面必须把“工具调用 ID → 工具执行结果”当作核心数据结构，而不是仅拼接文本。 ²⁷

批量调用与并行：执行器必须以数组语义实现

OpenAI 明确指出模型响应可能包含 0/1/多个 tool_calls，且“最佳实践是假设有多个”；并提供 parallel_tool_calls 来控制是否允许同一轮多个函数并行调用。LangChain 的工具调用说明同样包含 id 字段以将 call 与 result 关联。 ²⁸

工具面管理：控制工具数量与延迟加载

OpenAI 文档建议保持“初始可用函数数量小”，并给出“软建议：每轮开始时尽量少于 20 个函数”，必要时使用 tool_search 延迟加载不常用工具；同时工具定义会注入 system message、计入上下文并计费，因此工具描述过长会直接带来成本与效果风险。 ²⁹

错误、超时与可重试语义

工具封装必须把失败当作一等路径：对模型而言，“失败”不是异常栈，而是结构化可理解的状态机分支。MCP 规范把 error reporting、cancellation、progress tracking、logging 作为协议层的实用能力，提示工具系统应原生支持取消、进度与错误上报。 ³⁰

推荐的工具输出最小集合（适合 C++/微服务风格）：

- `status`: `ok` | `retryable_error` | `fatal_error` | `denied` | `timeout` | `invalid_args`
- `error_code`: 机器可判定的枚举（用于策略引擎/重试/告警）
- `error_message`: 面向开发者/审计的简短文本
- `data`: 成功时的结构化 payload
- `observability`: `trace_id/span_id`, `duration_ms`, `cached`, `attempt`

这些字段的治理价值在于：Planner/Executor 可据 `retryable_error` 自动重试或降级；审计系统可据 `denied` 回溯权限；告警系统可据 `fatal_error` 触发熔断与人工介入。其设计动机与 OWASP Top 10 中对“Model Denial of Service”“Excessive Agency”“Insecure Output Handling”等风险的关注一致：要用工程机制避免模型把错误当作可忽略文本继续“瞎编”。³¹

幂等性、重试与副作用隔离

Agent 的工具调用经常发生在不可靠网络、长时任务与可重试执行语义中，因此“幂等”与“副作用隔离”必须前置设计。

幂等 key：把“工具调用 ID”升级为业务幂等键

Stripe 的 API 文档指出 idempotency key 可用于安全重试，避免网络错误导致重复创建/更新。Temporal 文档也强调 Activities 可能被重复执行，推荐使用 idempotency keys 检测重复请求，并给出基于 run id / activity id 生成幂等键的建议。³²

实践建议：工具分类（Read / Write / Irreversible）

- Read-only：可缓存、可重试、无副作用
- Write-safe（可幂等）：必须支持幂等键；可重试
- Irreversible（不可逆）：默认禁止模型直接调用；必须人类审批/双重确认；或通过补偿事务实现可回滚

这类分级是落实“护栏”的工程化方式，契合 OpenAI 对“agent 在明确 guardrails 下运行”的强调，也契合 NIST AI RMF 对风险管理与治理角色的要求。³³

鉴权、审计、日志、指标与缓存

鉴权：把权限从 prompt 中移出，放到运行时上下文

工具签名里不应出现“你已经知道的参数”（如 user_id、tenant_id），而应由运行时注入；OpenAI 的函数设计建议指出应“尽量用代码卸载模型负担，不要让模型填写你已知的参数”。这既提升准确率，也降低提示注入导致越权的面。³⁴

审计与追踪：全链路 tracing 是可运维 Agent 的前提

OpenAI Agents SDK 以“保留完整 trace（包括工具调用、handoff、guardrails 等）”作为核心卖点之一，并在 SDK 文档中提供内置 tracing 能力。落到平台层面，OpenTelemetry 的 context propagation 默认采用 W3C TraceContext（traceparent 等 header），可用来把 agent run、工具调用与下游微服务纳入同一条分布式调用链。³⁵

缓存：两类缓存要区分

- Schema/工具定义缓存：OpenAI 文档指出 schema 会在首次请求进行额外处理并缓存，且 schema 变动会影响

延迟；因此工具版本要稳定、并为 schema 变更设计兼容策略。³⁶

- 结果缓存：面对高频 read-only 工具（如汇率、配置、知识库检索），可用 TTL/LRU 方式减少模型与工具调用次数，并降低 Model DoS 风险。³⁷

示例：工具 schema 样列表

下表给出一组“工程上常见、便于演示治理点”的工具样例（可直接作为你们团队的 schema 规范模板）。其关键在于：每个工具都明确副作用等级、幂等策略与权限域。³⁸

工具名	作用	副作用等级	必要输入 (JSON Schema 核心字段)	必要输出 (结构化)	幂等策略	鉴权建议
<code>search_docs</code>	文档检索	Read-only	<code>query</code> , <code>filters</code> , <code>top_k</code>	<code>hits[]</code> (含 <code>doc_id</code> 、片段、 <code>score</code>)	可缓存 (TTL)	以 tenant/ACL 注入过滤
<code>get_customer_profile</code>	获取客户资料	Read-only	<code>customer_id</code>	<code>profile</code>	可缓存 (短 TTL)	强制按租户校验 <code>customer_id</code>
<code>create_ticket</code>	创建工单	Write-safe (幂等)	<code>title</code> , <code>priority</code> , <code>context_ref</code>	<code>ticket_id</code>	必须 <code>idempotency_key</code>	角色校验 + 审计日志
<code>issue_refund</code>	退款	Irreversible	<code>order_id</code> , <code>amount</code> , <code>reason</code>	<code>refund_id</code> , <code>status</code>	必须幂等键 + 人工审批	强制审批/双人复核
<code>run_sql_readonly</code>	只读 SQL	Read-only	<code>query</code> , <code>params</code>	<code>rows</code> , <code>schema</code>	结果缓存/限流	数据域白名单、脱敏

工具调用工具的策略、风险与治理

“工具调用工具”本质上是编排 (orchestration) 问题：一次任务往往需要多工具、多步骤以及中间结果整合。OpenAI 明确指出模型可能在同一轮调用多个函数，并提供参数控制并行调用；Anthropic 则指出传统逐轮 tool calling 会导致中间结果污染上下文、并产生多次完整推理开销。³⁹

因此，是否允许工具互调，不应是“能不能”的问题，而是“以什么形式、在什么边界、用什么治理能力”的问题。

三种允许形态

固定编排 (fixed orchestration)

工具 A 内部按固定流程调用子工具（或子模块），对子工具的选择与顺序是确定的。这对应 OpenAI 函数设计建议里的“把总是顺序出现的函数合并”，用代码把常见序列固化，减轻模型在高频路径上的决策负担。 40

代码级组合 (programmatic composition)

在运行时层建立“组合工具” (composite tool) 或“pipeline tool”：它并不是自由决策，而是受控的 DAG/状态机。Haystack 的“Agent 使用 pipeline 作为工具”的方式可视为此类：组合逻辑在系统里可测试、可回放。 20

元工具/代码执行编排 (meta-tools)

让模型生成可执行代码（或声明式计划），由执行环境以代码方式调用多个工具、并在环境内处理大体量中间结果，最终把摘要/结果回传模型。Anthropic 的 Programmatic Tool Calling 明确提出该模式：通过 code execution 工具让 Claude 写 Python 编排逻辑、并在 sandbox 中执行，只有最终输出进入模型上下文，从而减少 token 污染并减少多次推理 round-trip，并可用 `asyncio.gather` 并行调用工具。 8

主要风险面

递归/失控调用链

一旦允许工具互调或 meta-tool 编排，最常见故障是“隐式递归”： $A \rightarrow B \rightarrow C \rightarrow A$ 或反复重试导致指数级调用，最终演化为 Model DoS（成本与延迟爆炸）。OWASP 在 LLM Top 10 中专门列出“Model Denial of Service”与“Excessive Agency”风险，提示应从设计上限制模型的可行动作与资源消耗。 41

追踪与调试困难

当工具互调发生在多层封装里，如果缺少 call graph 与 trace linkage，排障会退化为“读日志猜测”。OpenAI Agents SDK 把 tracing 作为核心能力，强调可记录工具调用与 handoffs 等事件；这类能力对工具互调尤为关键。 42

上下文污染与信息复制错误

Anthropic 指出传统逐轮 tool calling 会把大体量中间结果塞进上下文，挤掉重要信息；并且模型需要在自然语言里“目测数据、合并结果”，易错且慢。其 MCP 场景文章进一步举例说明：当工具规模达到数百/数千时，加载所有 tool definitions 和中间结果会显著推高 token 成本与出错概率。 43

安全边界被绕过

工具互调可能扩大攻击面：提示注入让模型诱导调用敏感工具、把机密带到外部、或通过工具组合实现原本禁止的操作。MCP 规范在 Security & Trust & Safety 中强调“用户同意与控制”等关键原则，这与企业落地的最小权限设计直接相关。 44

治理与缓解手段

预算系统 (Budgets) 应覆盖五个维度

- 最大步数/最大工具调用数 (step/tool budget)
- 最大深度 (max call depth: tool→tool→tool 上限)
- 最大 wall-clock (超时 + 取消)
- 最大 token/成本 (按模型与工具计费估算)
- 最大并行度 (并发上限, 防雪崩)

OpenAI 提供 `parallel_tool_calls` 控制同轮多工具；MCP 规范提供 cancellation/progress/logging 的协议语义；Temporal/Stripe 文档体系提供重试与幂等的工程范式。把这些组合起来才能形成“可控自治”。 45

白名单与 `allowed_callers`

Anthropic PTC 明确要求工具通过 `allowed_callers` opt-in 才能被 `code_execution` 编排调用；这是一种非常实用的“能力分层”做法：同一个工具可以允许 agent 直接调用，但禁止在 meta-tool 环境里调用，反之亦然。 46

可观测性优先：trace + call graph + 结构化事件

建议以 OpenTelemetry + W3C TraceContext 为底座，把每次 agent run 当 trace，每次模型调用/工具调用当 spans，并把 `tool_call_id`、user/tenant、工具名、预算消耗作为 span attributes。OpenTelemetry 的 context propagation 文档明确默认采用 W3C TraceContext；W3C 规范定义 traceparent 的字段结构，便于跨语言/跨服务传递。 47

安全基线：OWASP + NIST AI RMF 的落地化映射

OWASP Top 10 for LLM Applications 提供针对提示注入、不安全输出处理、模型 DoS、过度自治等风险分类；NIST AI RMF 则从治理、测量、管理等维度给出组织级风险管理框架。对 Agent 系统而言，应把二者落地到：权限与审批、内容与输出校验、资源与成本限额、供应链与依赖治理（工具/插件）。 48

运行时蓝图与接口示例

一个可生产化的 Agent Runtime 应把“大模型”视为可插拔策略模块，把“工具调用循环、状态管理、并发与重试、预算与权限、可观测性”视为平台能力。OpenAI 将 Responses API 与 Agents SDK 视为 agent building blocks，并强调可用 tracing/evaluations 评估 agent 性能；LangGraph 则强调 durable execution、human-in-the-loop 与 memory；Temporal 强调 crash-proof、可重试、长时执行。 49

推荐工程蓝图：模块、接口、数据流与限制

建议模块（C++ 为主，语言无关设计）：

- **Planner**：调用 LLM 产出下一步（工具调用/继续思考/请求人类/终止）
- **Executor**：执行 `tool_calls`（支持批量、并发、超时、重试、幂等）
- **Tool Registry**：工具元数据（schema、权限域、副作用等级、版本、超时、幂等策略）
- **State/Memory**：短期工作记忆（步骤与中间结果索引）、长期记忆（用户偏好/历史）、产物存储（artifacts）
- **Policy Engine**：预算、白名单、审批、敏感数据处理（脱敏/阻断）
- **Observability**：tracing（spans）、metrics（latency/cost/errors）、structured logs
- **Router/Tool Search（可选）**：当工具数大时，对工具面进行检索式加载（OpenAI `tool_search` / MCP 工具生态） 50

关键限制建议（默认值可作为 v1 基线）：

- 每轮可用工具：≤ 20（软建议），超出则用 tool search 或分域加载 40
- 最大步骤：20（交互式）/ 80（离线任务）
- 最大深度：2（工具互调）
- 并行：默认开启，但对不可逆工具强制串行 + 审批 51
- 严格 schema：`strict=true`（若平台支持），并把 schema 变更纳入版本治理 52

参考组件图 (Mermaid)

```
flowchart LR
    U[User / Upstream Service] -->|request| GW[Agent Gateway]
    GW --> RT[Agent Runtime]

    subgraph RT[Agent Runtime]
        PL[Planner\n(LLM client)]
        EX[Executor\n(tool runner)]
        TR[Tool Registry\n(schema, policy, timeouts)]
        ST[State & Memory\n(short/long, artifacts)]
        PE[Policy Engine\n(budgets, ACL, approvals)]
        OB[Observability\n(traces, metrics, logs)]
        CA[Cache\n(schema/results)]
    end

    PL <--> ST
    PL -->|tool calls| EX
    EX -->|lookup| TR
    EX -->|check| PE
    EX -->|read/write| CA
    EX -->|invoke| T1[(Tool A)]
    EX -->|invoke| T2[(Tool B)]
    EX -->|invoke| MCP[(MCP Server)]
    EX -->|tool outputs| PL

    RT --> OB
    GW --> OB
    T1 --> OB
    T2 --> OB
    MCP --> OB
```

该图的设计动机来自：OpenAI 的工具调用循环（Planner↔Executor）、MCP 把 Tools/Resources/Prompts 作为服务能力并提供 cancellation/logging、LangGraph 把 agent 视为可观测可恢复的长时状态机，以及 Temporal 的可重试执行语义。 ⁵³

参考时序图：一次标准工具调用回路 (Mermaid)

```
sequenceDiagram
    autonumber
    participant User
    participant Runtime as Agent Runtime
    participant Planner as Planner(LLM)
    participant Exec as Executor
    participant Tool as Tool Service
```

```

User->>Runtime: request(goal, context)
Runtime->>Planner: prompt + state + tool schemas
Planner-->>Runtime: tool_calls[] or final_answer
alt tool_calls returned
  Runtime->>Exec: execute(tool_calls[])
  loop for each tool_call (possibly parallel)
    Exec->>Tool: call(name, args, idempotency_key, auth_ctx)
    Tool-->>Exec: tool_result(status, data/error, call_id)
  end
  Exec-->>Runtime: tool_outputs[]
  Runtime->>Planner: append(tool_outputs) + updated_state
  Planner-->>Runtime: final_answer or more tool_calls
else final_answer returned
  Runtime-->>User: response
end

```

这一循环与 OpenAI 文档对 tool calling 的五步流程一致，并体现“支持多 tool_calls、按 call_id 关联输出”的关键约束。¹¹

JSON Function-Call 协议示例

下面给出一个“供应商中立但贴近主流 API 形态”的示例：请求里携带工具 schema；模型返回 tool_calls；应用执行后回传 tool_outputs 并再次请求模型生成最终结果。其设计要点来自 OpenAI 的 tool_calls/call_id 语义与 strict schema 建议，以及 Gemini 对每次 function call 唯一 id 的建议。⁵⁴

请求（向模型提供工具）

```

{
  "model": "any-tool-calling-llm",
  "input": [
    { "role": "system", "content": "你是客服自动化 Agent，必须在权限范围内操作。" },
    { "role": "user", "content": "帮我查询订单A123状态，如果已发货就把物流摘要发给我。" }
  ],
  "tools": [
    {
      "type": "function",
      "name": "get_order_status",
      "description": "按订单号查询订单状态（只读）",
      "strict": true,
      "parameters": {
        "type": "object",
        "additionalProperties": false,
        "properties": {
          "order_id": { "type": "string" }
        }
      }
    }
  ]
}

```

```

    },
    "required": ["order_id"]
  },
  {
    "type": "function",
    "name": "get_shipping_summary",
    "description": "按订单号获取物流摘要（只读）",
    "strict": true,
    "parameters": {
      "type": "object",
      "additionalProperties": false,
      "properties": {
        "order_id": { "type": "string" }
      },
      "required": ["order_id"]
    }
  }
],
"parallel_tool_calls": true
}

```

模型响应（发起工具调用，可能多个）

```

{
  "output": [
    {
      "type": "tool_call",
      "id": "call_001",
      "name": "get_order_status",
      "arguments": { "order_id": "A123" }
    }
  ]
}

```

应用回传工具结果

```

{
  "input": [
    {
      "type": "tool_output",
      "call_id": "call_001",
      "status": "ok",
      "data": { "order_id": "A123", "status": "shipped", "carrier": "SF",

```

```

"tracking_no": "SF123..." }
    }
  ]
}

```

模型第二轮（继续调用或直接输出）

```

{
  "output": [
    {
      "type": "tool_call",
      "id": "call_002",
      "name": "get_shipping_summary",
      "arguments": { "order_id": "A123" }
    }
  ]
}

```

C++ 与 Python 的接口/IDL 示例

以下示例倾向“少抽象、强契约、好观测”的风格，贴近 C++ 团队常用的服务端工程实践；其中“批量工具调用、call_id、幂等与超时、可取消、结构化错误”与 OpenAI/MCP/Temporal 等资料强调的约束对齐。 45

C++：工具接口 + 注册表 + 执行器（伪代码，可落地）

```

// Tool contract: structured result + idempotency + tracing
struct ToolCall {
    std::string call_id;           // generated by model or runtime
    std::string name;
    nlohmann::json arguments; // validated against JSON Schema
};

enum class ToolStatus { Ok, RetryableError, FatalError, Denied, Timeout,
InvalidArgs };

struct ToolResult {
    std::string call_id;
    ToolStatus status;
    nlohmann::json data;           // success payload
    std::string error_code;       // stable enum-like string
    std::string error_message; // short human-readable
    int64_t duration_ms = 0;
    bool cached = false;
    // observability

```

```

    std::string trace_id;
    std::string span_id;
};

struct ToolContext {
    std::string user_id;
    std::string tenant_id;
    std::string trace_id;
    std::string idempotency_key;    // required for write-safe tools
    std::chrono::milliseconds timeout;
    // cancellation token, auth claims, etc.
};

class ITool {
public:
    virtual ~ITool() = default;
    virtual std::string Name() const = 0;
    virtual nlhmann::json InputSchema() const = 0;    // JSON Schema
    virtual bool IsReadOnly() const = 0;
    virtual std::chrono::milliseconds DefaultTimeout() const = 0;

    // async call (thread pool / coroutine / future)
    virtual std::future<ToolResult> Call(const ToolCall& call, const ToolContext&
ctx) = 0;
};

class ToolRegistry {
public:
    void Register(std::shared_ptr<ITool> tool);
    std::shared_ptr<ITool> Get(const std::string& name) const;

    // policy metadata: scopes, side effects, allowed_callers, etc.
};

class ToolExecutor {
public:
    ToolExecutor(ToolRegistry* registry, PolicyEngine* policy, Observability*
obs);

    std::vector<ToolResult> ExecuteBatch(const std::vector<ToolCall>& calls,
                                         const ToolContext& ctx,
                                         bool allow_parallel) {
        // 1) validate (schema) + policy check (ACL/budget/side effects)
        // 2) run in parallel if allowed and safe
        // 3) enforce timeouts + retries for retryable errors
        // 4) emit spans + structured logs
        // 5) return results in call_id order
    }
};

```

```
}  
};
```

Python：工具装饰器 + Pydantic schema（方便生成 JSON Schema）

```
from pydantic import BaseModel, Field  
from typing import Literal, Any, Dict  
  
class GetOrderStatusArgs(BaseModel):  
    order_id: str = Field(..., description="订单号")  
  
class ToolResult(BaseModel):  
    call_id: str  
    status: Literal["ok", "retryable_error", "fatal_error", "denied",  
"timeout", "invalid_args"]  
    data: Dict[str, Any] = {}  
    error_code: str = ""  
    error_message: str = ""  
  
def get_order_status(args: GetOrderStatusArgs, ctx: dict) -> ToolResult:  
    # ctx contains user_id/tenant_id/auth/idempotency_key/trace_id  
    # validate auth, query DB/service, return structured result  
    ...
```

测试、监控、安全与运维，以及从 v1 到多 Agent 的迁移路径

把 Agent 做到“能跑”很容易，把 Agent 做到“可上线、可回归、可排障、可审计”才是核心难点。OpenAI 在其 agent 工具链中强调 tracing 与 evaluations；LangGraph 强调可视化与运行时指标；OWASP 与 NIST 提供安全与风险治理框架；Temporal 与 Stripe 提供可重试与幂等的工程范式。⁵⁵

测试策略

- 工具单元测试：对每个工具做输入 schema 校验、权限校验、错误码覆盖、超时与重试模拟。strict schema 能显著减少“参数形状漂移”。³⁶
- 合同测试（contract tests）：把工具 schema + 典型/边界输入作为固定测试向量，确保变更兼容；注意 OpenAI 提到 schema 会缓存，频繁变化会增加延迟与复杂度，因此更需要版本治理与回归。³⁶
- 端到端回放测试：记录 planner 的输入、模型输出的 tool_calls、工具结果、最终输出，形成可回放 trace；OpenAI Agents SDK 与其 tracing 文档强调能记录工具调用与事件。⁵⁶
- 长程鲁棒性评测：应特别关注“长对话/长任务一致性”；ToolHaystack 指出长程一致性仍是工具使用模型的短板，因此需要以架构护栏补上。²⁴

监控与观测

- 统一 trace 语义：以 OpenTelemetry 传播上下文，默认使用 W3C TraceContext；把 `agent_run_id`、`tool_call_id`、工具名、预算与重试次数作为 span attributes。 47
- 指标面板（核心）：成功率、重试率、超时率、每步 token、每任务工具调用数、并发度、P95 延迟、单位任务成本。OpenAI 提到工具定义与调用会产生 token 成本与计费影响，因此必须指标化。 57
- 结构化日志：不要只打自然语言日志；应按 `tool_result/status/error_code` 结构化，便于聚合告警。 58

安全与合规

- 风险模型：对照 OWASP Top 10 for LLM Applications，重点防 prompt injection、insecure output handling、model DoS、excessive agency、supply chain 等；在工具层落实最小权限、输出校验与资源限额。 59
- 组织治理：NIST AI RMF 强调在组织层面识别、评估与管理 AI 风险；对 Agent 系统可落地为：变更评审（工具/提示/策略）、权限与审批流程、数据处理规范、事故响应与审计。 60
- 协议与生态安全：MCP 规范强调强能力伴随安全与信任风险，并提出用户同意与控制等原则；在接入 MCP 工具生态时，应把授权范围（roots）、工具白名单与数据域隔离作为上线门槛。 61

从 v1 单 Agent 到多 Agent 的迁移路径

阶段一：v1（单 Agent + 工具治理）
目标是把 tool calling 的契约与运维体系做扎实：strict schema、批量 tool_calls、并发控制、预算、幂等、观测。OpenAI 文档对工具数量、token 成本、并行调用与 strict 模式给出了具体建议，可直接作为 v1 设计准则。 62

阶段二：v2（planner/executor + 工具路由）
引入显式规划与重规划，或引入 tool_search / 检索式工具加载，以应对工具面扩大与任务结构化需求。OpenAI 的 tool_search 与 Anthropic 关于工具定义膨胀的讨论，说明工具面路由是规模化的关键。 63

阶段三：v3（状态机/图 + durable execution）
当任务跨分钟/小时甚至跨天，需要失败恢复与人工介入时，应引入 LangGraph/Temporal 类 durable execution。LangGraph 强调持久化执行与 human-in-the-loop；Temporal 强调 crash-proof 执行与幂等重试。 5

阶段四：v4（多 Agent：handoff/manager）
在上述基础设施齐备后，再引入多 Agent 才能避免“协调成本吞噬收益”。OpenAI Agents SDK 将 handoffs、tools、tracing 作为 primitives；AutoGen 强调多 Agent 对话与可编程交互。 64

原型里程碑甘特图（Mermaid）

```
gantt
  title Agent 原型到可灰度上线的建议里程碑
  dateFormat YYYY-MM-DD
  axisFormat %m/%d

  section v1 单 Agent 闭环
  工具契约(JSON Schema/strict)与注册表 :a1, 2026-04-01, 10d
  执行器(批量tool_calls/并发/超时/重试/幂等) :a2, after a1, 12d
```

预算与权限(白名单/副作用分级/审批占位)	:a3, after a1, 10d
基础观测(结构化日志+tracing+核心指标)	:a4, after a2, 8d
section v2 规划与工具面扩展	
planner/executor 分离与重规划策略	:b1, after a4, 10d
工具路由/延迟加载(tool_search/分域工具集)	:b2, after b1, 10d
端到端回放测试与评测集	:b3, after b1, 10d
section v3 状态机与可恢复执行	
状态模型与持久化(会话/任务/产物)	:c1, after b3, 12d
durable execution/事件驱动(可取消/可恢复)	:c2, after c1, 14d
section v4 多 Agent	
handoff/manager 原语与协作协议	:d1, after c2, 12d
多 Agent 观测与成本治理	:d2, after d1, 10d

优先阅读清单与开源项目地图

本节按“基础概念 → 工具契约 → 编排与规模化 → 运行时与治理 → 安全与观测”的顺序给出阅读优先级，并尽量选取主来源（官方文档、标准、论文、开源仓库）。⁶⁵

优先阅读清单

Agent 与工具使用基础

- ReAct: 推理与行动交错的通用范式，解释了为何“工具交互”能降低幻觉与提升可解释性。⁶⁶
- OpenAI Function/Tool Calling 指南：strict schema、并行调用控制、tool_search、token 成本等关键工程细节（强烈建议逐行读）。⁶
- Gemini Function Calling 文档：对 function calling 的三类用例、以及 function call id 回传约束的清晰说明。⁶⁷

规划、记忆与更强 agent 行为

- Plan-and-Solve Prompting：规划-执行的提示层范式，可用于对 planner 模块做启发式设计。¹⁵
- Tree of Thoughts：把推理视为搜索/回溯，帮助理解“需要规划/搜索”的任务为何单循环不稳。⁶⁸
- Reflexion：用语言反馈形成“episodic memory”，为自我改进/反思型 agent 提供思路。⁶⁹

多 Agent 与编排

- AutoGen：多 Agent 对话框架的代表性论文（工程抽象与交互可编程性）。⁷⁰
- Anthropic Programmatic Tool Calling：meta-tool 编排的工业化方案，重点关注 token 污染与多轮推理开销问题。⁸
- LangGraph Overview：把 agent 作为长时有状态图/状态机，并强调 durable execution 与 human-in-the-loop。⁷¹

标准化生态与安全治理

- MCP Specification：工具/资源/提示的协议化、JSON-RPC 2.0、能力协商、取消/进度/日志/错误上报，以及安全原则。⁶¹

- OWASP Top 10 for LLM Applications：面向 LLM/Agent 的安全威胁地图，适合作为安全评审基线。 59
- NIST AI RMF 1.0（及 GenAI Profile）：组织级 AI 风险治理框架，可用于制度化上线流程与审计。 72
- W3C Trace Context + OpenTelemetry Context Propagation：为 agent 系统落地分布式追踪提供标准基础。 73

可靠性工程

- Temporal Durable Execution 与 Activity Idempotency：把“可重试工具调用”工程化为 Activities，并用幂等键防重复副作用。 23

关键开源项目与参考实现

- OpenAI Agents SDK (Python)：提供 agent primitives (tools、handoffs、tracing)，适合研究“最小抽象如何支撑可运维多步骤 workflow”。 74
- Semantic Kernel：模型无关 SDK，强调插件/工具与编排，并明确从 planners 转向 function calling 的路线。 75
- LangChain / LangGraph 文档：工具调用事件结构 (name/args/id)、runtime 概念、以及面向长时状态机的 orchestration 能力。 76
- Haystack Agents：展示 agent 如何把 pipeline 当工具，适合研究“固定流程 + agent 决策”的混合形态。 20
- Langfuse（观测与评测）：开源 LLM/Agent observability 平台，适合研究 traces/cost/latency/evals 的工程落地。 77
- CrewAI（多 Agent 编排框架）：适合了解“角色化多 Agent”在社区中的常见建模方式与边界问题。 78

以上项目与文档共同指向同一个趋势：Agent 成功与否越来越取决于运行时与治理能力（契约、预算、可恢复、可观测、安全），而不是提示词技巧本身。 79

1 5 22 71 LangGraph overview - Docs by LangChain

<https://docs.langchain.com/oss/python/langgraph/overview>

2 33 Agents • Cookbook

<https://developers.openai.com/cookbook/topic/agents/>

3 6 9 11 14 25 26 27 28 29 34 36 38 39 40 45 50 51 52 53 54 57 62 63 65 Function calling | OpenAI API

<https://developers.openai.com/api/docs/guides/function-calling/>

4 12 13 66 ReAct: Synergizing Reasoning and Acting in Language Models

https://arxiv.org/abs/2210.03629?utm_source=chatgpt.com

7 76 Tool calling - Docs by LangChain

https://docs.langchain.com/oss/python/langchain/frontend/tool-calling?utm_source=chatgpt.com

8 43 46 Introducing advanced tool use on the Claude Developer Platform \ Anthropic

<https://www.anthropic.com/engineering/advanced-tool-use>

10 19 35 64 Agents SDK | OpenAI API

<https://developers.openai.com/api/docs/guides/agents-sdk/>

15 **Plan-and-Solve Prompting: Improving Zero-Shot Chain-of-Thought Reasoning by Large Language Models**

https://arxiv.org/abs/2305.04091?utm_source=chatgpt.com

16 17 **What are Planners in Semantic Kernel | Microsoft Learn**

<https://learn.microsoft.com/en-us/semantic-kernel/concepts/planning>

18 70 **AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation**

https://arxiv.org/abs/2308.08155?utm_source=chatgpt.com

20 21 **Build a Tool-Calling Agent | Haystack**

https://haystack.deepset.ai/tutorials/43_building_a_tool_calling_agent

23 **The definitive guide to Durable Execution**

https://temporal.io/blog/what-is-durable-execution?utm_source=chatgpt.com

24 **ToolHaystack: Stress-Testing Tool-Augmented Language ...**

https://arxiv.org/html/2505.23662v1?utm_source=chatgpt.com

30 44 58 61 **Specification - Model Context Protocol**

<https://modelcontextprotocol.io/specification/2025-11-25>

31 37 41 48 59 **OWASP Top 10 for Large Language Model Applications**

https://owasp.org/www-project-top-10-for-large-language-model-applications/?utm_source=chatgpt.com

32 **Idempotent requests | Stripe API Reference**

https://docs.stripe.com/api/idempotent_requests?utm_source=chatgpt.com

42 56 **Tracing - OpenAI Agents SDK**

https://openai.github.io/openai-agents-python/tracing/?utm_source=chatgpt.com

47 **Context propagation**

https://opentelemetry.io/docs/concepts/context-propagation/?utm_source=chatgpt.com

49 55 79 **New tools for building agents | OpenAI**

<https://openai.com/index/new-tools-for-building-agents/>

60 72 **Artificial Intelligence Risk Management Framework (AI RMF 1.0)**

https://nvlpubs.nist.gov/nistpubs/ai/nist.ai.100-1.pdf?utm_source=chatgpt.com

67 **Function calling with the Gemini API | Google AI for Developers**

<https://ai.google.dev/gemini-api/docs/function-calling>

68 **Tree of Thoughts: Deliberate Problem Solving with Large Language Models**

https://arxiv.org/abs/2305.10601?utm_source=chatgpt.com

69 **Reflexion: Language Agents with Verbal Reinforcement Learning**

https://arxiv.org/abs/2303.11366?utm_source=chatgpt.com

73 **Trace Context**

https://www.w3.org/TR/trace-context/?utm_source=chatgpt.com

74 **openai/openai-agents-python: A lightweight, powerful ...**

https://github.com/openai/openai-agents-python?utm_source=chatgpt.com

75 **microsoft/semantic-kernel: Integrate cutting-edge LLM ...**

https://github.com/microsoft/semantic-kernel?utm_source=chatgpt.com

77 langfuse/langfuse: Open source LLM engineering ...

https://github.com/langfuse/langfuse?utm_source=chatgpt.com

78 crewAIInc/crewAI: Framework for orchestrating role-playing ...

https://github.com/crewaiinc/crewai?utm_source=chatgpt.com